1. Padrão Singleton

Cenário: Gerenciador de Configurações Globais

Problema:

Uma aplicação corporativa precisa carregar as configurações do sistema (URLs de banco de dados, chaves de API, timeouts) a partir de um arquivo JSON/YAML durante a inicialização. Múltiplos módulos da aplicação (autenticação, relatórios, pagamentos) precisam ler essas configurações frequentemente ao longo da execução do programa.

Desafio:

1. Garantir que o arquivo de configuração seja lido e carregado na memória **apenas uma única vez**.

2. Fornecer um **ponto de acesso global** e único para essas configurações.

Exercício:

1. Modele e implemente a classe `ConfiguracaoSistema` usando o padrão **Singleton**.

2. Torne o construtor privado e crie um método de acesso (ex: `getInstance()`).

3. Implemente uma aplicação que irá solicitar várias vezes as configurações. Apresente evidências de que existe apenas uma instância e um único ponto de acesso às informações de configuração.

2. Padrão Adapter

Cenário: Integração de Gateway de Pagamento Terceirizado

Problema:

Seu sistema de e-commerce já possui uma interface padronizada para processar pagamentos chamada `ProcessadorPagamento`, com o método `processarPagamento(double valor, String numeroCartao)`. A empresa decidiu integrar um novo provedor de pagamentos chamado **PayFast**, fornecido por uma biblioteca externa (`PayFastSDK.jar`).

Desafio:

A classe `PayFastService` da biblioteca de terceiros possui métodos e assinaturas completamente diferentes: `efetuarCobranca(BigDecimal quantia, String cartaoCredito, String moeda)`. Você **não pode alterar o código da biblioteca de terceiros** e não quer alterar o código do seu e-commerce que já depende de `ProcessadorPagamento`.

Exercício:

- Identifique os papéis no padrão: **Target** (Alvo), **Adaptee** (Adaptado) e **Adapter** (Adaptador).
- Crie a classe `PayFastAdapter` que implementa a interface `ProcessadorPagamento` e encapsula uma instância de `PayFastService`.
- Faça a conversão necessária de tipos e parâmetros dentro do método adaptado (ex: converter `double` para `BigDecimal` e passar a moeda padrão `"BRL"`).
- **Questão para reflexão:** Qual é a diferença entre o **Adapter de Objeto** (composição) e o **Adapter de Classe** (herança)? Qual deles é mais recomendado e por quê?

3. Padrão Facade (Fachada)

Cenário: Sistema de Reserva de Pacotes de Viagem

Problema:

Em um sistema de viagens, para concluir a reserva de um pacote, o código do cliente (ex: o controlador da página web) precisa interagir diretamente com vários subsistemas complexos e independentes:

- `ServicoVoo`: busca assentos e efetua reserva de passagens.
- `ServicoHotel`: verifica disponibilidade e reserva quartos.
- `ServicoAluguelCarro`: reserva veículos na cidade de destino.
- `ServicoPagamento`: processa a cobrança do valor total.
- `ServicoNotificacao`: envia confirmação por e-mail/SMS.

Desafio:

O código do cliente está extremamente poluído e acoplado a todos esses 5 subsistemas, tendo que gerenciar a ordem correta das chamadas e o tratamento de erros de cada um.

Exercício:

- Crie a classe de fachada `FachadaReservaViagem` (ou `TravelBookingFacade`).
- Implemente um método simplificado de alto nível: `reservarPacoteCompleto(DadosCliente cliente, DadosViagem viagem)`.
- Dentro desse método, orquestre as chamadas para os métodos dos subsistemas na ordem adequada.
- Demonstre como o cliente agora interage apenas com uma única chamada simples para concluir a operação.
- **Questão para reflexão:** O padrão Facade impede que o cliente acesse diretamente os subsistemas caso precise de uma personalização avançada?

4. Padrão Composite

Cenário: Gerenciador de Arquivos e Pastas (Sistema de Arquivos)

Problema:

Você está desenvolvendo um componente de sistema de arquivos para um sistema operacional ou aplicação desktop. O sistema possui dois tipos principais de elementos:

- **Arquivo:** representa um arquivo individual (ex: `.pdf`, `.png`), possui nome e tamanho em bytes, e não pode conter outros elementos dentro dele.
- **Pasta:** possui um nome e pode conter tanto arquivos quanto outras pastas (subpastas).
- **Desafio:** O sistema precisa calcular o tamanho total em disco ocupado por qualquer item. Se o item for um arquivo, seu próprio tamanho é retornado. Se for uma pasta, o tamanho total é a soma do tamanho de todos os arquivos e subpastas contidos nela. O cliente deve ser capaz de tratar arquivos e pastas de forma **uniforme**, sem precisar usar checagens manuais do tipo `if (item instanceof Pasta)`.

Exercício:

- Modele a interface ou classe abstrata comum `ComponenteFileSystem` (com o método `getTamanho()`).
- Implemente a classe folha `Arquivo` (*Leaf*).
- Implemente a classe composta `Pasta` (*Composite*), mantendo uma lista interna de `ComponenteFileSystem` e os métodos `adicionar(ComponenteFileSystem c)` e `remover(ComponenteFileSystem c)`.
- Implemente o método `getTamanho()` na classe `Pasta` de forma recursiva.
- **Desafio extra:** Adicione um método `exibirEstrutura(String indentacao)` para imprimir no console a árvore completa de diretórios e arquivos formatada visualmente.